深度学习笔记

Attention Is All You Need

Attention Is All You Need

目录

学习笔记: Attention Is All You Need

—

1. 研究背景与问题定义

1.1 这个领域之前存在什么问题?

在 Transformer 出现之前，** 序列转导模型（Sequence Transduction Models）** 主要基于循环神经网络架构，具体包括：

RNN/LSTM/GRU 的固有缺陷：

| 问题类型 | 具体表现 | |———|———| | ** 顺序计算的瓶颈 ** | 隐藏状态 h_t 必须等待 h_{t-1} 计算完成，导致训练时无法并行化 | | ** 长距离依赖问题 ** | 信号在网络中传播的路径长度为 $O(n)$，距离越远，梯度消失/爆炸问题越严重 | | ** 内存限制 ** | 批处理受到序列长度的严重限制，长序列无法有效批训练 | | ** 计算效率低下 ** | 训练时间随序列长度线性增长 |

正如论文所述：> ”The fundamental constraint of sequential computation, however, remains.”

1.2 本文要解决什么具体问题?

本文提出解决 ** 如何在保持高质量翻译的同时，大幅提升序列模型的并行计算效率 ** 的问题。

具体目标：

- 消除循环层（RNN/LSTM/GRU）带来的顺序依赖
- 用 ** 注意力机制（Attention Mechanism）** 完全替代循环结构
- 在机器翻译任务上达到或超越当时最优的模型质量
- 显著减少训练时间

1.3 为什么这个问题重要?

1. ** 并行化是深度学习扩展的关键 **：GPU/TPU 的并行计算能力无法被 RNN 充分利用
2. ** 翻译系统的实际应用需求 **：需要更快的训练和推理速度
3. ** 长距离依赖是 NLP 的核心挑战 **：RNN 难以有效建模远距离语义关系
4. ** 该方法具有通用性 **：解决此问题的方法可能适用于其他序列建模任务

—

2. 相关工作梳理

2.1 之前的工作做了什么?

| 模型 | 年份 | 主要贡献 | 核心架构 | |——|——|————|————| | **Seq2Seq + Attention** | 2014 | 首次将注意力机制引入 NMT | RNN + Bahdanau Attention | | **GNMT** | 2016 | Google 大规模 NMT 系统 | 8 层 LSTM + Attention | | **ConvS2S** | 2017 | 使用卷积替代 RNN | CNN + Multi-hop Attention | | **ByteNet** | 2017 | 线性时间复杂度翻译 | Dilated CNN | | **Neural GPU** | 2016 | 可并行化的计算模型 | 卷积架构 | | **Self-Attention** | - | 用于阅读理解、摘要等任务 | Attention-only |

2.2 它们的局限性

RNN-based 模型（GNMT 等）：

- 无法并行化：时间步必须顺序处理
- 长距离依赖学习困难
- 训练时间极长（数周）

CNN-based 模型（ConvS2S, ByteNet）：

- 虽然可并行，但建立远距离依赖需要多层堆叠
- ConvS2S：路径长度 $O(n)$
- ByteNet：路径长度 $O(\log_k(n))$（使用膨胀卷积）
- 计算成本随卷积核大小增加

** 传统注意力机制：**

- 大多作为 RNN 的附加组件，而非核心
- 无法独立完成序列建模

2.3 本文与它们的区别

| 特性 | 本文方法 (Transformer) | 传统方法 (RNN/CNN) | |——|————————-|————————-| | ** 核心架构 ** | 纯注意力机制 | RNN/CNN + 注意力 | | ** 路径长度 ** | $O(1)$ 常数 | $O(n)$ 或 $O(\log(n))$ | | ** 并行化程度 ** | 完全并行 | 部分并行 | | ** 长距离依赖 ** | 直接建模 | 需要多层/门控 | | ** 训练效率 ** | 12 小时（8 GPU）| 数天至数周 |

—

3. 核心方法详解

3.1 整体架构

Transformer 采用经典的 **Encoder-Decoder** 结构，但完全用注意力机制实现：

“‘ 输入序列 (x , ..., x) ↓ [Encoder] 6 层堆叠 ↓ 连续表示 z = (z , ..., z) ↓ [Decoder] 6 层堆叠 ↓ 输出序列 (y , ..., y) “‘

Transformer 整体架构图示：

“‘ Encoder Stack (N=6 layers) Input Embedding + Positional Encoding Layer 1: Multi-Head Self-Attention + Feed-Forward (Residual+LN) Layer 2: (same structure)... ... Layer 6: (same structure) ↓ Decoder Stack (N=6 layers) Output Embedding + Positional Encoding Layer 1: Masked Self-Attention + Encoder-Decoder Attention + Feed-Forward (Residual+LN) Layer 2-6: (same structure)... Linear + Softmax “‘

3.2 Encoder 组件详解

3.2.1 编码器结构 每个编码器层包含两个 ** 子层（Sub-layer）**：

“‘python

伪代码表示

class EncoderLayer: def __init__(self): self.self_attention = MultiHeadAttention() self.feed_forward = PositionwiseFFN() self.norm1 = LayerNorm() self.norm2 = LayerNorm()

def forward(self, x):

子层 1：多头自注意力 + 残差连接

attn_output = self.self_attention(x) x = self.norm1(x + attn_output)

子层 2：前馈网络 + 残差连接

ff_output = self.feed_forward(x) x = self.norm2(x + ff_output)

return x “‘

** 残差连接公式：**

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

3.3 Decoder 组件详解

解码器层包含 ** 三个子层 **：

| 子层 | 功能 | 特殊处理 | |——|——|———| | Masked Multi-Head Self-Attention | 自注意力，防止看到未来位置 | **Masking** | | Multi-Head Encoder-Decoder Attention | 关注编码器输出 | Q 来自前一层，K,V 来自 Encoder | | Position-wise Feed-Forward Network | 非线性变换 | 同 Encoder |

**Masking 机制 **（保持自回归性质）：

- 将” 非法” 注意力权重设置为 $-\infty$
- 确保位置 i 的预测只依赖位置 $< i$ 的已知输出

3.4 注意力机制详解

3.4.1 Scaled Dot-Product Attention（缩放点积注意力） ** 核心思想 **：将注意力视为 ** 查询（Query）** 对 ** 键值对（Key-Value）** 的检索。

** 输入 **：

- Query: $\mathbf{Q} \in \mathbb{R}^{n \times d_k}$
- Key: $\mathbf{K} \in \mathbb{R}^{m \times d_k}$
- Value: $\mathbf{V} \in \mathbb{R}^{m \times d_v}$

** 公式 **：

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

** 计算流程图解：**

“‘ Query Q (n×dk) Key K (m×dk) Value V (m×dv)

QK^T / √dk ← 计算点积并缩放 Softmax ← 归一化得到注意力权重 × V ← 加权求和 Output (n×dv) “‘

** 为什么需要缩放（Scaling）？ **

当 d_k 较大时，点积的值会变得很大，导致 softmax 进入饱和区，梯度极小。

数学解释：

- 假设 q 和 k 的分量是均值 0、方差 1 的独立随机变量
- 则 $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ 的均值为 0，方差为 d_k
- 除以 $\sqrt{d_k}$ 可将方差归一化为 1

3.4.2 Multi-Head Attention（多头注意力） ** 设计动机 **:

- 单一注意力头只能关注一种" 表示子空间"
- 不同注意力头可以学习不同的相关性模式
- 允许模型同时关注不同位置的不同表示子空间

** 公式 **:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, ..., \text{head}_h)\mathbf{W}^O$$

其中:

$$\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V)$$

** 投影矩阵维度 **:

$$\mathbf{W}_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad \mathbf{W}_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad \mathbf{W}_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}, \quad \mathbf{W}^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$$

** 本文配置 **:

- $h = 8$ 个并行注意力头
- $d_k = d_v = d_{\text{model}}/h = 512/8 = 64$
- 总计算量与单头全维度注意力相似

** 多头注意力可视化: **

"‘ Input Q, K, V (d_model=512) [W ^Q ... W ^Q] ← 8 个不同的 Query 投影 [W ^K ... W ^K] ← 8 个不同的 Key 投影 [W ^V ... W ^V] ← 8 个不同的 Value 投影 Attention Attention ... Attention Head 1 Head 2 ... Head 8 ← 并行计算

Concat (8 × 64 = 512)

W^O ← 最终投影 Output (d_model=512) "‘

3.4.3 三种注意力应用 | 位置 | Query 来源 | Key/Value 来源 | 作用 | |——|———-|————–|——| | **Encoder Self-Attention** | Encoder 前一层 | Encoder 前一层 | 建模输入序列内部依赖 | | **Decoder Self-Attention** | Decoder 前一层（带 Mask）| Decoder 前一层 | 建模输出序列内部依赖（仅过去）| | **Encoder-Decoder Attention** | Decoder 前一层 | Encoder 最终输出 | 对齐源序列与目标序列 |

3.5 Position-wise Feed-Forward Networks

每个位置独立应用相同的两层全连接网络:

$$\text{FFN}(x) = \max(0, x\mathbf{W}_1 + b_1)\mathbf{W}_2 + b_2$$

** 等效于核大小为 1 的两个卷积 **:

- 内层维度：$d_{ff} = 2048$（base 模型）
- 激活函数：ReLU
- 外层维度：$d_{\text{model}} = 512$

3.6 Positional Encoding（位置编码）

由于模型不含循环和卷积，需要 ** 显式注入位置信息 **。

** 正弦/余弦位置编码 **：

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

** 设计动机 **：

- 波长从 2π 到 $10000 \cdot 2\pi$ 的几何级数
- 可以表示任意固定偏移 k 的相对位置：PE_{pos+k} 可线性表示为 PE_{pos} 的函数
- 可处理训练时未见过的更长序列

** 位置编码图示：**

“‘ 位置 → 维度编码 0 1 2 3 4 5 6 7 (d_model 维度) pos=1 [sin,cos,sin,cos,sin,cos,sin,cos] pos=2 [sin,cos,sin,cos,sin,cos,sin,cos] pos=3 [sin,cos,sin,cos,sin,cos,sin,cos] ... “‘

3.7 复杂度分析

| 层类型 | 每层复杂度 | 顺序操作数 | 最大路径长度 | |——|————|————|————-| | Self-Attention | $O(n^2 \cdot d)$ | $O(1)$ | $O(1)$ | | Recurrent | $O(n \cdot d^2)$ | $O(n)$ | $O(n)$ | | Convolutional | $O(k \cdot n \cdot d^2)$ | $O(1)$ | $O(\log_k(n))$ |

** 关键洞察 **：当序列长度 n 小于表示维度 d 时（这在大多数实际应用中常见），自注意力的计算效率高于循环层。

3.8 训练细节

3.8.1 优化器 使用 **Adam 优化器 **，配合 ** 学习率预热策略 **：

$$lrate = d_{\text{model}}^{-0.5} \cdot \min\left(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5}\right)$$

** 预热阶段 **：4000 步内线性增加学习率 ** 衰减阶段 **：按步数的平方根倒数衰减

3.8.2 正则化

1. ** 残差 Dropout**：$P_{drop} = 0.1$
2. 对每个子层输出应用 dropout
3. 对 embedding 与 positional encoding 之和应用 dropout

1. ** 标签平滑 **：$\epsilon_{ls} = 0.1$
2. 改善困惑度但提升 BLEU

3.8.3 模型变体配置

模型	N	d_{model}	d_{ff}	h	d_k	d_v
Base	6	512	2048	8	64	64
Big	6	1024	4096	16	64	64

---

4. 实验设计与结果分析

4.1 数据集和评估指标

** 数据集：**

任务	句子对数量	词表大小	分词方法
WMT 2014 English-German	~4.5M	37,000	BPE
WMT 2014 English-French	~36M	32,000	Word-piece

** 评估指标：**

- **BLEU (Bilingual Evaluation Understudy)**：机器翻译的标准评估指标

** 训练设置：**

- 硬件：8 NVIDIA P100 GPUs
- Base 模型：每步 0.4 秒，训练 100,000 步（~12 小时）
- Big 模型：每步 1.0 秒，训练 300,000 步（~3.5 天）

4.2 主实验结果

4.2.1 WMT 2014 English-to-German

模型	BLEU	训练成本 (FLOPs)
ByteNet	23.75	-
GNMT + RL	24.6	2.3×10^1
ConvS2S	25.16	9.6×10^1
MoE	26.03	2.0×10^1
Deep-Att + PosUnk Ensemble	40.4	8.0×10^2
GNMT + RL Ensemble	26.30	1.8×10^2
ConvS2S Ensemble	26.36	7.7×10^1
Transformer (base)	**27.3**	**3.3×10^1 **
Transformer (big)	**28.4**	**2.3×10^1 **

** 关键发现 **：

- Big 模型比所有先前最佳模型（包括集成）** 高 2.0 BLEU**
- 训练成本 ** 仅为 GNMT 的约千分之一 **

4.2.2 WMT 2014 English-to-French | 模型 | BLEU | 训练成本 (FLOPs) | |——|——|——————| | GNMT + RL | 39.92 | 1.4×10^2 | | ConvS2S | 40.46 | 1.5×10^2 | | MoE | 40.56 | 1.2×10^2 | | ConvS2S Ensemble | 41.29 | 1.2×10^{21} | | **Transformer (base)** | **38.1** | **3.3×10^1 ** | | **Transformer (big)** | **41.8** | **2.3×10^1 ** |

** 关键发现 **:

- Big 模型建立 ** 新的单模型最高 BLEU 记录 41.8**
- 训练成本仅为最佳模型的 ** 约 1/4**

4.3 消融实验分析

4.3.1 注意力头数和维度的影响（表 3-A） | 配置 | h | $d_k = d_v$ | PPL | BLEU | |——|—|————-|——|——| | base | 8 | 64 | 4.92 | 25.8 | | 1 头 | 1 | 512 | 5.29 | 24.9 | | 4 头 | 4 | 128 | 5.00 | 25.5 | | 16 头 | 16 | 32 | 5.01 | 25.4 | | 32 头 | 32 | 16 | 5.01 | 25.4 |

** 分析 **:

- 单头注意力比最佳配置低 0.9 BLEU
- 过多注意力头反而降低性能
- 8 头是性能和效率的平衡点

4.3.2 Key 维度的影响（表 3-B） | d_k | PPL | BLEU | |——-|——|——| | 64 | 4.92 | 25.8 | | 32 | 5.16 | 25.1 | | 16 | 5.01 | 25.4 |

** 分析 **：减小 key 维度会损害模型质量，暗示兼容性函数比简单点积更复杂可能有益。

4.3.3 模型大小的缩放（表 3-C） | N | d_{model} | d_{ff} | 参数 (M) | PPL | BLEU | |—|————————-|————-|————|——|——| | 2 | 256 | 1024 | 28 | 6.11 | 23.7 | | 4 | 256 | 1024 | 50 | 5.19 | 25.3 | | 4 | 512 | 1024 | 80 | 4.88 | 25.5 | | 6 | 1024 | 4096 | 168 | 4.66 | 26.0 |

** 分析 **：更大的模型和更深/更宽的网络都能带来性能提升。

4.3.4 Dropout 的影响（表 3-D） | Dropout | PPL | BLEU | |————|——|——| | 0.0 | 5.77 | 24.6 | | 0.1 | 4.92 | 25.8 | | 0.2 | 4.95 | 25.5 |

** 分析 **：Dropout 对防止过拟合至关重要，0.1 是最佳值。

4.3.5 位置编码比较（表 3-E） | 位置编码 | BLEU | |————-|——| | Sinusoidal | 25.8 | | Learned | 25.7 |

** 分析 **：两种方法性能几乎相同，选择正弦波是因为可以外推到更长序列。

4.4 泛化实验：English Constituency Parsing

| Parser | 设置 | F1 | |———|——|——| | Transformer (4 layers) | WSJ only | **91.3** | | Vinyals & Kaiser et al. | WSJ only | 88.3 | | Petrov et al. | WSJ only | 90.4 | | Dyer et al. | WSJ

only | 91.7 | | Transformer (4 layers) | semi-supervised | **92.7** | | Dyer et al. | semi-supervised | 93.3 |

** 关键发现 **:

- 仅用 WSJ 数据（40K 句子），Transformer 超越大多数模型
- 证明 Transformer 可迁移到其他序列任务
- 特别在小数据场景表现优异

—

5. 优缺点评价

5.1 本文的主要优点

1. ** 革命性的架构创新 **
2. 首次完全用注意力机制替代循环/卷积结构
3. 打破了 RNN 在序列建模中的垄断地位

1. ** 卓越的并行计算效率 **
2. 训练时间从数周缩短到数小时
3. 可充分利用 GPU/TPU 的并行能力

1. ** 有效建模长距离依赖 **
2. 任意位置间的路径长度为常数 $O(1)$
3. 解决了 RNN 的梯度消失问题

1. ** 高质量翻译性能 **
2. WMT EN-DE: 28.4 BLEU（超越所有集成模型）
3. WMT EN-FR: 41.8 BLEU（新纪录）
4. 泛化能力强，可用于其他 NLP 任务

1. ** 优雅且可解释的设计 **
2. 多头注意力可可视化
3. 不同注意力头学习不同任务
4. 可解释性强

5.2 本文的局限性

1. ** 计算复杂度随序列长度平方增长 **
2. 自注意力的复杂度为 $O(n^2 \cdot d)$
3. 对于极长序列（如文档、音频）计算成本过高
4. 论文提到可用" 受限注意力"（$O(r \cdot n \cdot d)$）缓解

1. ** 位置编码的局限 **
2. 正弦/余弦编码是手工设计，可能非最优
3. 学习得到的相对位置表示可能更有效
4. （后续工作如 BERT 使用了学习的位置嵌入）

1. ** 全局信息的代价 **
2. 缺乏局部性的归纳偏置
3. 对于需要局部模式的任务（如音频、图像），可能需要更多参数或数据
4. （后续工作如 Vision Transformer 也证明了其可行性）

5.3 可能的改进方向

1. ** 层次化注意力 **：处理超长序列
2. ** 相对位置编码 **：如 Transformer-XL、RoPE 等
3. ** 稀疏/线性注意力 **：如 Linformer、Performer 等
4. ** 融合局部和全局注意力 **：如 BigBird 等
5. ** 跨模态扩展 **：用于图像、语音、视频等

—

6. 个人思考与启发

6.1 这篇论文对我的研究有什么启发?

1. ** 注意力机制是序列建模的核心 **
2. 注意力不仅是一种" 技巧"，而是足以支撑整个模型的核心机制
3. 应当深入理解注意力机制的数学本质和优化特性

1. ** 简化可以带来突破 **

2. 去除复杂的 RNN 结构，回归简单的注意力机制
3. ” 简单即美” 的设计哲学
4. 有时看似” 复古” 的方法配合现代优化技术可以焕发新生

1. ** 理论分析与工程实践的结合 **
2. 对路径长度、计算复杂度的理论分析
3. 为实践选择提供指导

6.2 可以借鉴的思想和技术

1. **Multi-Head Attention 设计 **
2. 多个注意力头捕获不同特征
3. 分治思想的应用

1. ** 残差连接 + LayerNorm**
2. 解决深层网络的训练困难
3. 已成为现代深度学习的标准组件

1. ** 学习率预热策略 **
2. 对 Transformer 等深层模型尤为重要
3. 可推广到其他深层网络

1. ** 标签平滑 **
2. 改善泛化能力的简单技巧

1. ** 模型变体的系统消融分析 **
2. 科学的研究方法
3. 逐步验证每个组件的贡献

6.3 可能的 Follow-up 研究方向

| 方向 | 问题 | 可能的改进 | |——|——|————| | ** 高效注意力 ** | $O(n^2)$ 复杂度 | Linformer, Performer, FlashAttention | | ** 更好的位置编码 ** | 相对位置建模 | RoPE, ALiBi, T5 Relative Attention | | ** 更通用的 Transformer** | 视觉、语音任务 | ViT, Speech Transformer | | ** 更深更宽的模型 ** | 扩展性 | MoE, Switch Transformer | | ** 轻量化模型 ** | 部署效率 | DistilBERT, MobileBERT, TinyBERT | | ** 预训练 + 微调范式 ** | 迁移学习 | BERT, GPT, T5 |

6.4 论文的历史意义

这篇论文的重要性远超其原始任务——机器翻译。它奠定了 ** 现代 NLP 的基石 **：

- **BERT (2018)**：使用 Transformer 编码器进行预训练
- **GPT 系列 (2018-2023)**：使用 Transformer 解码器
- **T5 (2019)**：统一所有 NLP 任务为文本到文本
- **ViT (2020)**：将 Transformer 扩展到计算机视觉
- ** 大语言模型 (2020-至今)**：GPT-3, PaLM, LLaMA 等

可以说，**"Attention is All You Need" 这篇论文开启了深度学习的新时代 **。

—

7. 关键词与核心概念

| 关键词/概念 | 简要解释 | |————|————| | **Transformer** | 完全基于注意力机制的序列到序列模型架构 | | **Self-Attention** | 输入序列内部各位置之间的注意力，用于建模内部依赖关系 | | **Scaled Dot-Product Attention** | 缩放点积注意力，通过 Query-Key-Value 机制计算注意力权重 | | **Multi-Head Attention** | 多头注意力，多个并行注意力机制以捕获不同子空间的信息 | | **Encoder-Decoder Architecture** | 编码器-解码器结构，先编码输入再解码输出 | | **Positional Encoding** | 位置编码，注入序列位置信息以弥补模型对位置的不感知 | | **Residual Connection** | 残差连接，缓解深层网络的训练困难 | | **Layer Normalization** | 层归一化，稳定训练过程 | | **Feed-Forward Network** | 前馈神经网络，位置-wise 的非线性变换 | | **Masked Attention** | 掩码注意力，防止解码器看到未来信息 | | **BLEU** | Bilingual Evaluation Understudy，机器翻译评估指标 | | **Sequence Transduction** | 序列转导，将一个序列转换为另一个序列的任务 | | **Auto-regressive** | 自回归，生成时依赖之前生成的符号 | | **Parallelization** | 并行化，同时处理多个计算任务 | | **Long-range Dependency** | 长距离依赖，相距较远的元素之间的依赖关系 | | **Word-piece/Subword** | 子词分词方法，处理未登录词问题 | | **Beam Search** | 束搜索，翻译推理时的解码策略 | | **Label Smoothing** | 标签平滑，正则化技术 | | **Adam Optimizer** | 自适应矩估计优化器 | | **Learning Rate Scheduling** | 学习率调度策略 |

—

附录：核心公式汇总

1. **Scaled Dot-Product Attention**

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

1. **Multi-Head Attention**

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, ..., \text{head}_h)\mathbf{W}^O$$

1. **Position-wise FFN**

$$\text{FFN}(x) = \max(0, x\mathbf{W}_1 + b_1)\mathbf{W}_2 + b_2$$

1. **Residual Connection + Layer Normalization**

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

1. **Learning Rate Schedule**

$$lrate = d_{\text{model}}^{-0.5} \cdot \min\left(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5}\right)$$

1. **Sinusoidal Positional Encoding**

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

—

> ** 笔记总结 **：这篇论文是深度学习领域的里程碑工作，首次证明了”注意力机制”足以支撑强大的序列建模能力，摒弃了传统的循环结构。Transformer 不仅在机器翻译任务上取得了突破性的成果，更重要的是，它开创了现代 NLP 的新时代，为后续 BERT、GPT 等划时代模型奠定了基础。